

# MC809 Computação Distribuída

## Laboratório 5

Nome: Arthur Costa Barreto

RA: 206325

Data: Outubro de 2026

### 1. Introdução

#### 1.1. Descrição da Proposta do Laboratório

Este laboratório consiste no treinamento de Split-Learning de clientes em paralelo utilizando o formato U-shaped e o dataset CIFAR-10. Nesse formato de treinamento, cada cliente possui um conjunto próprio de imagens que passam por camadas iniciais de ativação do seu próprio modelo e depois enviam as ativações dessas camadas iniciais para o servidor. No servidor, a maior parte do modelo vive e a parte mais pesada do treinamento acontece: mais camadas e camadas maiores. Após passar por essas camadas, o servidor envia o seu conjunto de ativações para os clientes, que possuem os labels dos dados. Por fim, os clientes calculam a loss do modelo e o processo se inverte.

Esse formato de treinamento é muito utilizado, pois os dados nunca passam pelo servidor, o que reduz a exposição dos dados durante a comunicação entre cliente e servidor. Por outro lado, essa privacidade extra é paga pela necessidade de 4 comunicações entre cliente e servidor a cada batch: 2 durante o forward pass e 2 durante o backpropagation. Isso implica maior latência no treinamento e um custo de transmissão de dados na rede.

#### 1.2. Divisão dos arquivos

Para este exercício, existem 4 arquivos principais:

- [client.py](#): Arquivo que contém a definição da classe Client (actor do Ray), constrói o modelo inicial e final e implementa as funções de treino e teste;
- [server.py](#): Arquivo que contém a definição da classe servidor e o modelo intermediário.
- [splitlearning.proto](#): Arquivo que contém a estrutura da mensagem comunicada entre cliente e servidor. Para este laboratório, ele contém 3 serviços que serão descritos em mais detalhes posteriormente.
- [display\\_results.ipynb](#): Notebook apenas de visualização dos dados de treino e teste;

### 2. Construção do modelo e treinamento de Split-Learning

#### 2.1 Separação do modelo

Para esse problema, foi construída uma rede convolucional onde 2 camadas convolucionais estão no cliente, intercaladas por MaxPooling2D e BatchNormalization; 2 camadas convolucionais estão no servidor, também intercaladas por MaxPooling2D e BatchNormalization, seguidas por um Flatten, uma camada densa de 256 neurônios e uma ativação ReLU. Por fim, o cliente tem a parte final do modelo com uma camada densa de 128

neurônios, seguido por uma ativação ReLU e um softmax do tamanho da saída (10 neurônios).

## 2.2 Comunicação entre cliente e servidor

Ao todo, existem 4 momentos de comunicação entre cliente e servidor:

1. O cliente passa a imagem pelas camadas convolucionais iniciais do modelo e envia ao servidor suas ativações (repeated float), o tamanho do batch de imagens (int32), o id do cliente (int32) e o id do batch.
2. O servidor passa as ativações enviadas pelo cliente pelas camadas convolucionais intermediárias do modelo e envia a mesma estrutura de mensagem da primeira comunicação de volta ao cliente.
3. O cliente recebe o vetor de features do servidor e passa pelas camadas densas antes de aplicar o softmax. Em seguida, calcula a loss entre a resposta predita e os rótulos e envia ao servidor o gradiente (repeated float) da loss retropropagada pela camada densa em relação às ativações enviadas na comunicação 2, o tamanho do batch de imagens, o id do cliente e o id do batch.
4. O Servidor recebe os gradientes do modelo final do cliente e faz o processo de retropropagação, atualizando os pesos, e envia ao modelo inicial do cliente o gradiente da loss retropropagada em relação às ativações enviadas na comunicação 1.

Para comportar essas 4 comunicações, 2 serviços são criados no arquivo .proto - além de um extra para o teste. No primeiro serviço, o cliente envia as ativações iniciais (1) e recebe as ativações do servidor (2). No segundo serviço, o cliente envia os gradientes do modelo final (3) e recebe os gradientes do servidor (4). O serviço extra de teste utiliza a mesma estrutura de mensagem do treino, mas o servidor executa o modelo em modo de inferência: a BatchNormalization usa as médias acumuladas no treino sem atualizá-las, e não são guardados gradientes nem atualizados pesos.

## 2.3 Estratégia de treino

Para o treinamento do modelo de forma paralela, utilizou-se o FedAvg para atualizar os pesos dos modelos parciais dos clientes. Essa é uma forma de atualização de pesos em que os clientes partem de um modelo inicial igual e, a cada época, atualizam os pesos de forma ponderada com a quantidade de dados utilizados durante o treino. Esse processo consiste em os modelos utilizarem a loss e gradientes calculados para o seu conjunto próprio de dados e atualizar os pesos de forma local. Ao final de cada época de treino, a média ponderada entre os valores dos pesos atualizados é calculada e cada cliente recebe novamente um modelo igual para que, na próxima época, o processo se repita.

O modelo do servidor é único e compartilhado: ele é atualizado pelos gradientes de todos os clientes conforme chegam, com um lock serializando o acesso.

Para além disso, utilizou-se a `sparse_categorical_crossentropy` como função de perda, a `sparse_categorical_accuracy` como métrica de avaliação e estipulou-se um batch de 64 amostras para o treinamento, num total de 10 épocas. Vale mencionar também que optou-se por um `learning_rate` de  $1e-3$  para os modelos no cliente e servidor com otimizador Adam, e

houve um pré-processamento leve nos dados de treino e teste, ao dividir o valor dos pixels por 255 – os deixando na faixa entre 0 e 1 para evitar problemas na atualização dos pesos.

## 2.4 Visualização do modelo e do treinamento Split-Learning

**Split Learning U-Shaped: M1 e M3 no cliente, M2 no servidor (comunicação via gRPC)**

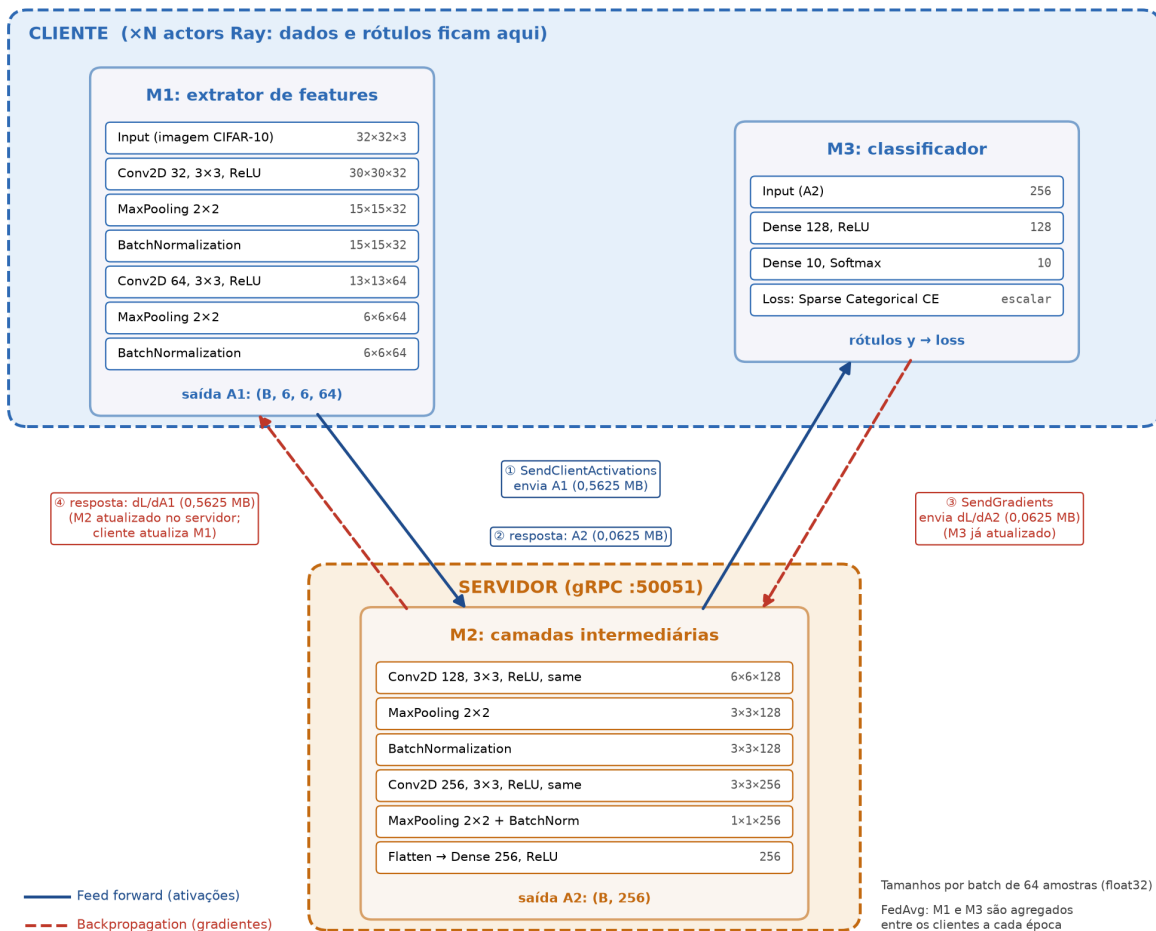


Figura 1: Separação dos modelos do cliente e servidor em treinamento U-shaped

## 3. Resultados

Apesar da dificuldade de atualizar os pesos de clientes em paralelo, devido à necessidade de ponderar atualizações individuais sobre conjuntos de dados diferentes, podemos verificar no plot abaixo que o objetivo foi atingido: loss cai e acurácia sobe com o passar das épocas, apesar de haver desvios pontuais durante os batches de treinamento.

Também vemos que a latência se mantém quase o tempo todo dentro de uma mesma faixa (0,2s a 0,1s). No entanto, vemos um pico no primeiro batch que pode ser explicado pelo cold start da GPU – os primeiros dados transferidos entre CPU e GPU tomam mais tempo do que o restante da comunicação.

Por fim, analisa-se um volume por batch constante (0,625 MB enviados: A1 com 0,5625 MB e dL/dA2 com 0,0625 MB), pois o tamanho dos tensores depende apenas da arquitetura e do tamanho do batch. Por isso, o volume acumulado cresce linearmente,

chegando a 1.625 MB por cliente ao fim das 10 épocas ( $260 \text{ batches} \times 10 \times 0,625 \text{ MB}$ ). Na escala logarítmica do gráfico, esse crescimento linear aparece como uma curva que achata.

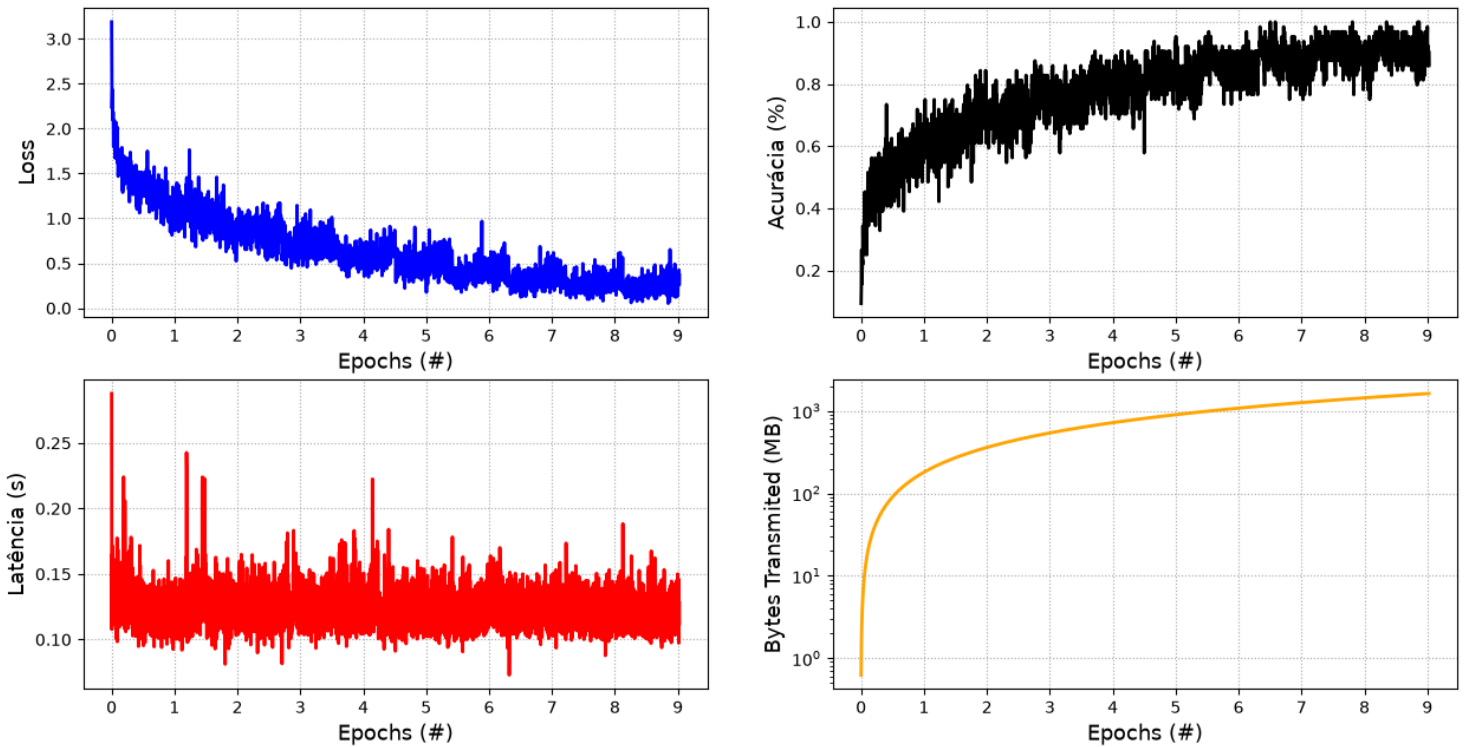


Figura 2: Métricas de treinamento do cliente 0 (loss, acurácia, latência e Megabytes transmitidos)

Para além do já comentado, os dois plots a seguir mostram a comparação entre a loss e acurácia dos 3 clientes utilizados no experimento com o passar das épocas de treino. Novamente vemos que, conforme o esperado, a loss cai e acurácia sobe com o passar das épocas e, apesar de pequenas inconsistências, os valores de loss e acurácia praticamente não divergem entre os clientes.

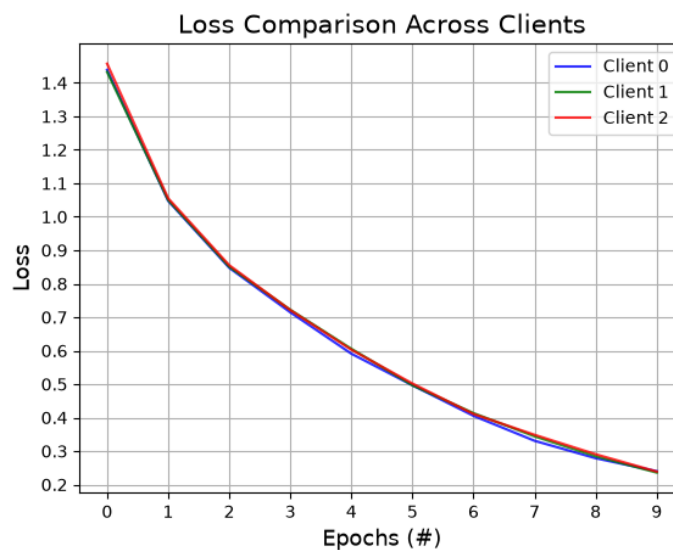


Figura 3: Loss individual dos 3 clientes X épocas

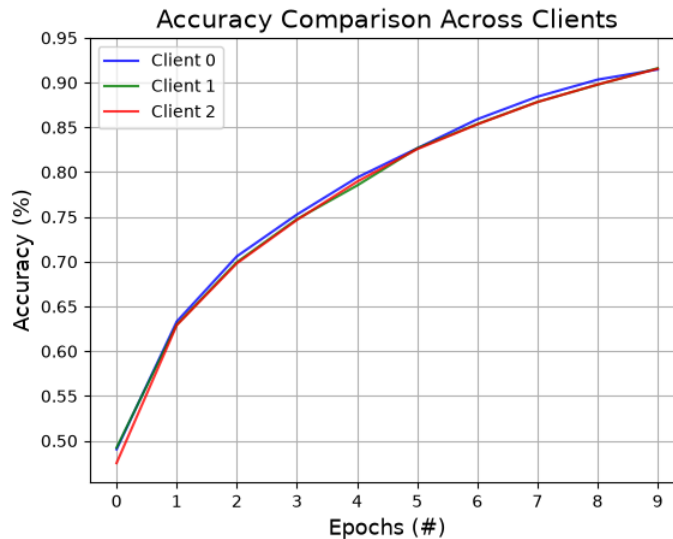


Figura 4: Acurácia individual dos 3 clientes X épocas

Por fim, os dados de teste foram separados em subconjuntos de mesmo tamanho e divididos entre os 3 clientes, que compartilham o mesmo modelo. O resultado foi de 71,53% de acurácia média com desvio padrão de 0,005967 entre os subconjuntos, atingindo uma latência média de 0,011788s, bem abaixo do que foi visto em treino.

#### 4. Conclusão

O Split-Learning U Shaped se mostrou uma ótima opção para treinar modelos com privacidade, já que não há necessidade de enviar os dados de treino ao servidor. No entanto, isso é pago com o custo de comunicação, principalmente durante o treinamento, já que é preciso enviar dados na predição e no backpropagation. Durante a inferência esse custo é reduzido, pois só há necessidade de 2 comunicações entre cliente e servidor e não é necessário calcular gradientes nem atualizar pesos, o que também reduz o tempo de espera pelo lock no servidor. Essa diminuição de latência é vista no resultado do teste, onde o valor médio foi quase 10 vezes menor do que o valor da faixa de latência em treino.

Além disso, treinar clientes em paralelo de forma distribuída se mostrou um desafio na medida em que cada cliente possui um modelo local que deve ser sincronizado com os outros clientes a fim de treinar o modelo global com todos os dados possíveis. Isso foi feito neste laboratório com a técnica do FedAvg, que se mostrou eficaz para manter os clientes sincronizados, como pode ser visto nos plots de loss e acurácia com os dados de treino dos 3 clientes.

Apesar disso, nota-se que o resultado do modelo está claramente sofrendo de overfitting, já que o valor da acurácia em teste está 20 pontos percentuais abaixo da acurácia de treino na última época. Isso ocorreu porque faltaram técnicas de regularização (Dropout, data augmentation etc) durante o treinamento, o que levou o modelo a memorizar parte do conjunto de treino.